Universidade Estadual do Oeste do Paraná
Campus Cascavel
Centro de Ciências Exatas e Tecnológicas
Curso de Bacharelado em Ciência da Computação

TRABALHO DE COMPILADORES – PARTE 1

Requisitos para a Especificação/Implementação da LP
Implementação do Analisador Léxico

Luiz Eduardo Garzon de Oliveira
Pedro Henrique Berti de Oliveira
Weberson Morelli Leite Junior

Cascavel, junho de 2025

1.1 Requisitos para a especificação/implementação da LP

```
-----------------------------------------------------------------------------------------------------
1. Nome da Linguagem: TRUCODE
-----------------------------------------------------------------------------------------------------


-----------------------------------------------------------------------------------------------------
2. Tipos de dados primitivos suportados:

    - int     (OURO)   -> Números inteiros.
    - float   (ESPADA) -> Números reais.
    - string  (COPA)   -> Textos.
    - boolean (PAUS)   -> Verdadeiro ou Falso.
-----------------------------------------------------------------------------------------------------


-----------------------------------------------------------------------------------------------------
3. Operadores Suportados:

    A. Operadores Lógicos:
        - AND   (##)
        - OR    (??)
        - NOT   (XX)

    B. Operadores Relacionais:
        - <     (RECUA)
        - >     (MATA)
        - ==    (BATE)
        - <=    (SEGURA)
        - >=    (CANTA)
        - !=    (BLEFA)

    C.  Operadores Aritméticos:
        - +     (KING)
        - -     (JACK)
        - /     (QUEEN)
        - *     (AS)
        - %     (RESTO)
-----------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------------------------------------------------------
4. Formação dos Identificadores (Regra para Nomes de Variáveis):

    - Expressão Regular : [a-zA-Z]([a-zA-Z0-9_])*
    - Tamanho Mínimo    : 1 caractere.
    - Tamanho Máximo    : 12 caracteres.
    - Não podem ser palavras reservadas.
    - Case Sensitive.

    EXEMPLOS:
        - Declaração                    : Nome-da-Variavel(Tipo da Variável)#
        - Declaração e Atribuição       : Nome-da-Variavel(Tipo da Variãvel) VALE valor#
-----------------------------------------------------------------------------------------------------

-----------------------------------------------------------------------------------------------------
5. Comandos de Entrada e Saída (Um para Entrada e Um para Saída):

    - Entrada   : RECEBE -> RECEBE[identificador(tipo)]#
    - Saída     : MOSTRA -> MOSTRA[valor_ou_texto, identificador(tipo)]#
-----------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------------------------------------------------------
6. Estruturas Suportadas pela Linguagem (Decisão, Repetição, etc.) e seus Respectivos Comandos.

    - Estrutura de Decisão (IF - ELSE):

        TRUCO(Condition) {
            ...
        }
        CORRE {
            ...
        }

    - Laco de Repetição Condicional (WHILE):

        SEGUE(Condition) {
            ...
        }

    - Laco de Repetição Contado (FOR):

        DISTRIBUI identificador DE inicio ATE fim PASSO valor {
            ...
        }
-----------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------------------------------------------------------
7. Palavras-Reservadas.

    - Comandos de E/S:
        1. RECEBE
        2. MOSTRA

    - Tipos de Dados Primitivos:
        3. OURO     (int)
        4. ESPADA   (float)
        5. COPA     (string)
        6. PAUS     (boolean)

    - Operadores Lógicos:

    - Operadores Relacionais:
        7.  RECUA   (<)
        8.  MATA    (>)
        9.  BATE    (==)
        10. SEGURA  (<=)
        11. CANTA   (>=)
        12. BLEFA   (!=)

    - Operadores Aritméticos:
        13. KING    (+)
        14. JACK    (-)
        15. QUEEN   (/)
        16. AS      (*)
        17. RESTO   (%)

    - Estruturas de controle:
        18. TRUCO       (if)
        19. CORRE       (else)
        20. SEGUE       (while)
        21. DISTRIBUI   (for)
        22. DE          (Início no Laço Contado)
        23. ATE         (Limite de Laço Contado)
        24. VALE        (Atribuição, =)
        25. CORTA       (Break)
```

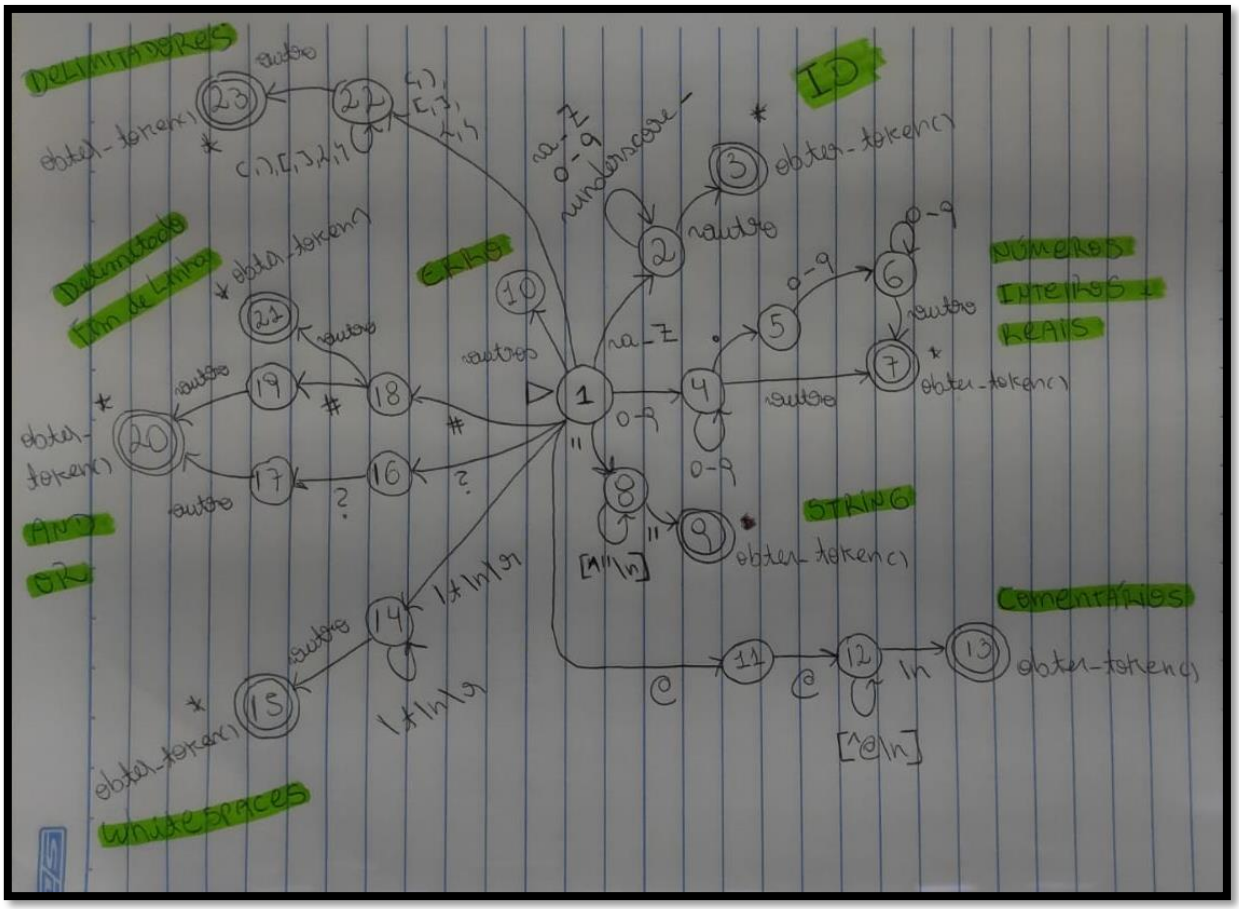
```
    - Constantes booleanas:
        26. REAL    (True)
        27. FAKE    (False)
-----------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------------------------------------------------------
8. Operadores Simbólicos:

    - Operadores Lógicos:
        - AND   (##)
        - OR    (??)
        - NOT   (XX)

    - Delimitadores e Símbolos Especiais:
        - @@  (Comentário de Linha)
        - #   (Delimitar de Fim de Linha, ;)
-----------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------------------------------------------------------
9. Estrutura Geral do Programa (Sintaxe):


----------------------------------------------
EXEMPLO 1:
----------------------------------------------
@@ Declaração de Variáveis
limite(OURO) VALE 0#
counter(OURO)#

@@ Entrada e Saída de Dados
MOSTRA["Informe um valor: "]#
RECEBE[limite]#

@@ Estrutura Condicional
TRUCO(limite MATA 0) {

    @@ Laco de Repeticao Contado
    DISTRIBUI counter DE 0 ATE limite PASSO 1 {
        MOSTRA[counter]#
    }
}
CORRE {
    MOSTRA["O limite deve ser maior que 0: "]#
}
```

```
-----------------------------------------------
EXEMPLO 2:
-----------------------------------------------
numA(OURO)#
numB(OURO)#
numC(ESPADA)#
resultado(ESPADA)#

RECEBE[numA]#
RECEBE[numB]#
RECEBE[numC]#

@@ Expressão Aritmética
resultado VALE ((numA KING numB) JACK (numC QUEEN 2))#

MOSTRA["O resultado vale: ", resultado]#

TRUCO(resultado CANTA 0) {
    MOSTRA["O resultado é maior ou igual a zero."]#
}
CORRE {
    MOSTRA["O resultado é menor que zero."]#
}
```

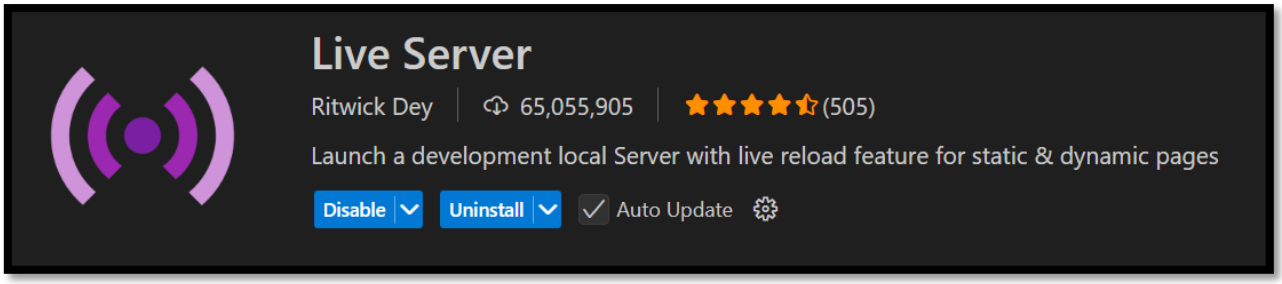
```
-----------------------------------------------
EXEMPLO 3:
-----------------------------------------------
resultado(PAUS)#

@@ Expressão com Operadores Relacionais.
resultado VALE XX(((10 BLEFA 10) ## (5 RECUA 10)) ?? (100 MATA 50))#

MOSTRA["O resultado é: ", resultado]#

-----------------------------------------------
EXEMPLO 4: TOKENS
-----------------------------------------------
@@ Definindo nome do jogador
nome(COPA)#
RECEBE[nome]#

TOKENS GERADOS:
    - <COMENTARIO, "Definindo nome do jogador">
    - <ID, nome>
    - <(>
    - <TIPO, COPA>
    - <)>
    - <#>
    - <RECEBE>
    - <[>
    - <ID, nome>
    - <]>
    - <#>
-----------------------------------------------
```

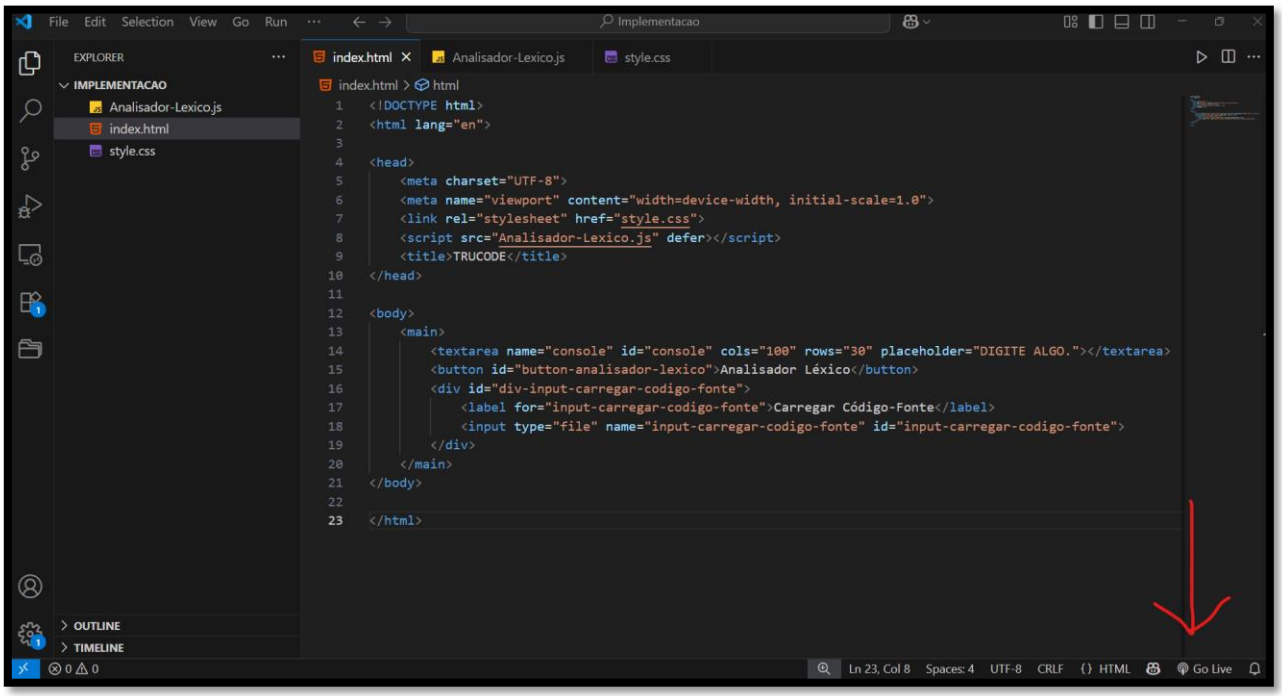

DELIMITADORES
ID
Delimitador
Fim de Linha
CHAR
NÚMEROS INTEIROS e REAIS
STRING
AND
OR
Comentários
White spaces
obter_token()
a-z
0-9
underscore
auto
0-9
a-z
0-9
0-9
[^"\n]
[^@\n]
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

1.2 Implementação do Analisador Léxico

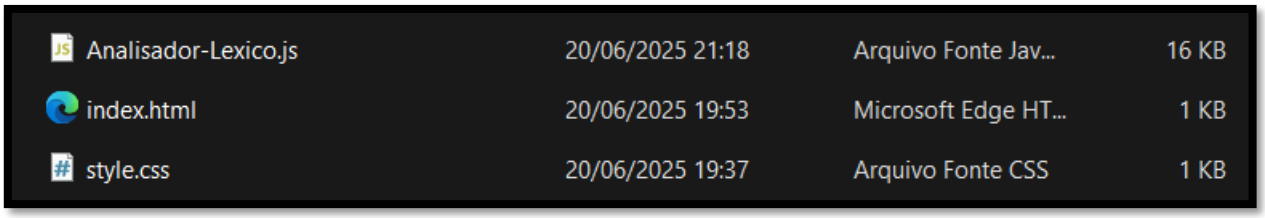
Modos de execução:

1. **O analisador léxico foi implementado em Javascript e pode ser executado utilizando a extensão Live Server no VSCODE.**

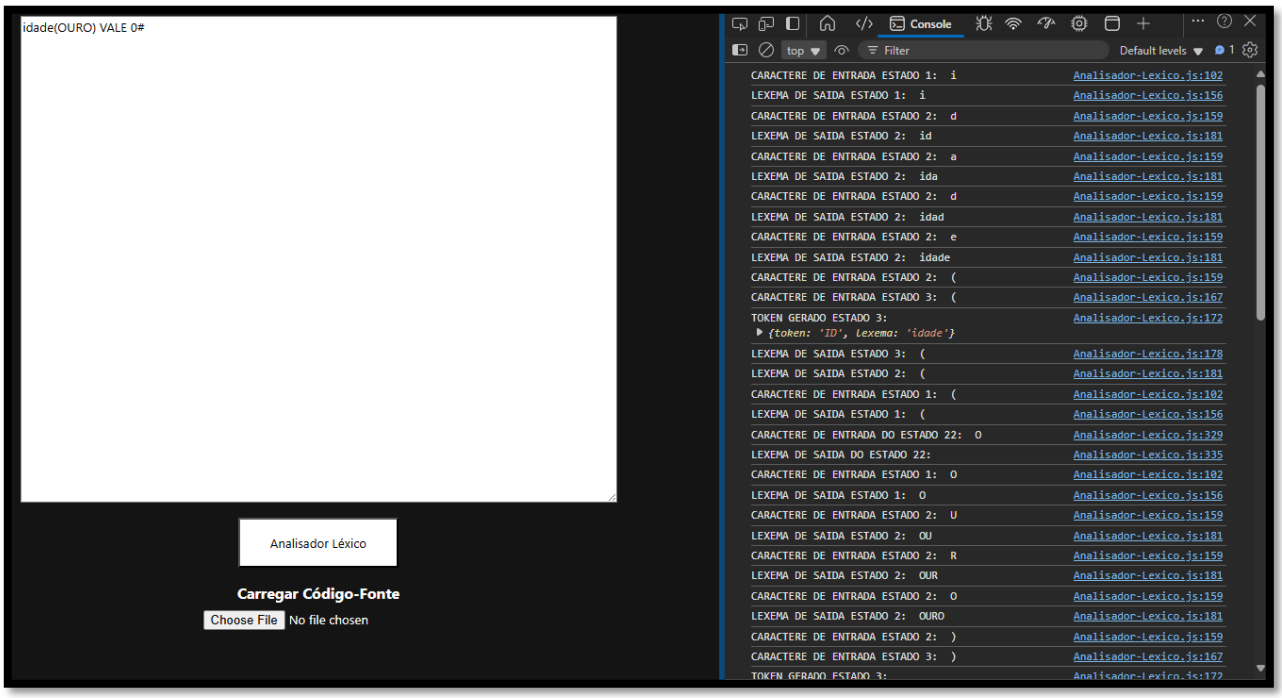
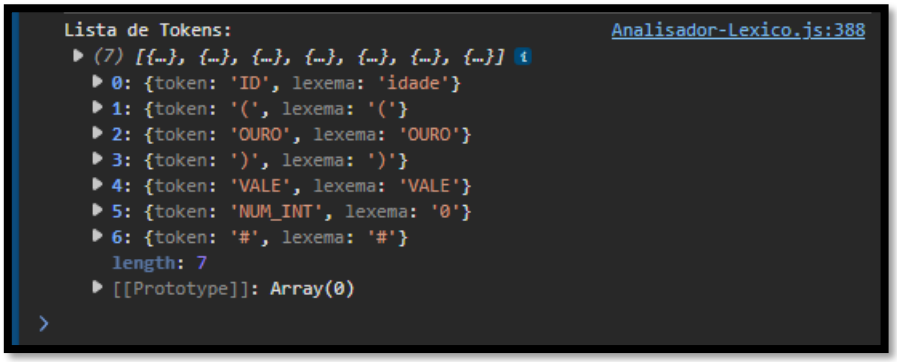




2. **Clicando duas vezes sobre o index.html.**



Analisador-Lexico.js	20/06/2025 21:18	Arquivo Fonte Jav...	16 KB
index.html	20/06/2025 19:53	Microsoft Edge HT...	1 KB
style.css	20/06/2025 19:37	Arquivo Fonte CSS	1 KB

A aplicação permite escrever diretamente o código-fonte na área de texto e depois executar análise léxica clicando no botão "Analisador Léxico" ou carregar e executar diretamente um arquivo de código fonte clicando em "Choose File".

No console do navegador é possível observar todas as operações que ocorrem durante a análise léxica bem como os tokens que são gerados.